

■ Table of Contents

#	Section	Page
1	Project Overview & Purpose	3
2	Tech Stack — What & Why	4
3	System Architecture — How It All Connects	5
4	Project Folder Structure	6
5	Database Schema — All 14 Tables	7
6	Backend — Flask API Deep Dive	9
7	Frontend — All 11 Pages Explained	13
8	Authentication Flow — Step by Step	17
9	Interview Flow — End to End	18
10	Anti-Cheat System	20
11	AI Service — Questions & Feedback	21
12	PDF Report Generation	22
13	Deployment Architecture	23
14	Environment Variables Reference	24
15	How to Make Changes — Developer Guide	25
16	Common Interview Questions & Answers	27

1. Project Overview & Purpose

What is it? The AI Resume Interview Mock Portal is a full-stack web application that helps students practice mock job interviews. It generates personalized MCQ questions based on the student's resume and target job role, conducts a timed interview in two rounds, and then provides detailed AI feedback on performance.

The Problem It Solves:

- Students don't know what questions to expect for their skill set
- No way to practice under real timed interview conditions
- No structured feedback on performance after practice
- No guidance on which skills are missing from their resume
- No roadmap for what to study next

How It Solves Them:

- Parses the student's uploaded resume using NLP to extract skills, projects, and experience
- Generates relevant MCQ questions via Claude AI tailored to the job role and resume
- Conducts a 2-round timed interview (Technical + HR) with anti-cheat enforcement
- Calculates detailed scores and generates 7-section AI feedback
- Creates a downloadable PDF report with full results and improvement roadmap

Project Name	AI Resume Interview Mock Portal
Developer	Roni Seikh
Live URL	https://ai-interview-portal-roni.vercel.app/
Backend URL	https://ai-interview-portal-glsn.onrender.com/
GitHub	https://github.com/Roni-Seikh/ai-interview-portal
Frontend Host	Vercel (free forever)
Backend Host	Render (free tier — sleeps after 15 min)
Database Host	Clever Cloud MySQL (free, 10MB)
AI Service	Anthropic Claude API + rule-based local fallback

2. Tech Stack — What & Why

Frontend

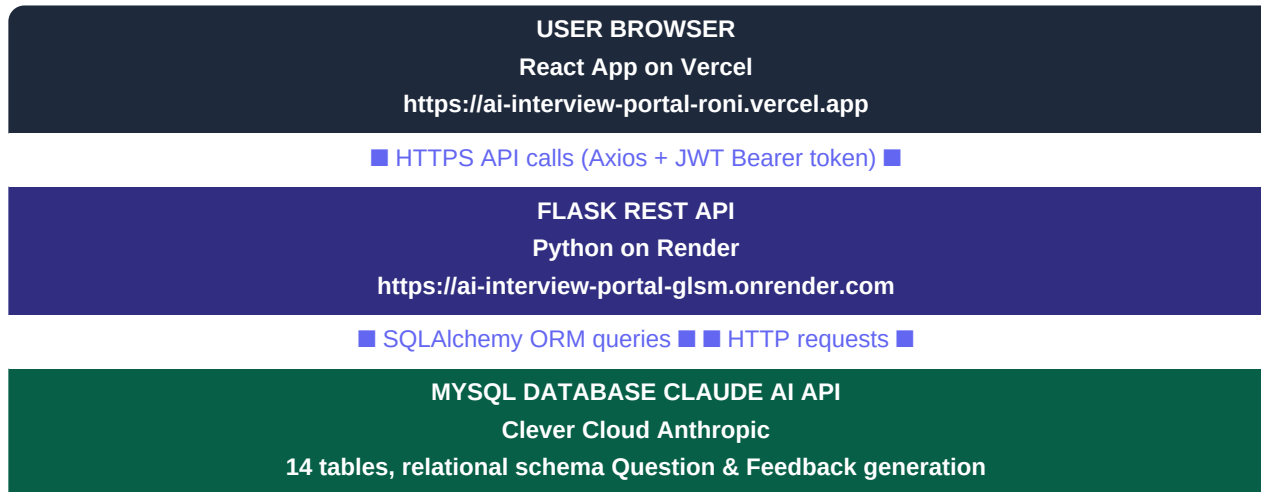
Technology	Version	Purpose	Why Chosen
React	18	UI framework	Component-based, fast re-renders, huge ecosystem
Tailwind CSS	3	Styling	Utility-first, no CSS files, consistent dark theme
Framer Motion	11	Animations	Smooth page transitions and micro-interactions
React Router	6	Routing	Client-side navigation, protected routes
Axios	1.7	HTTP client	Auto JWT injection, token refresh on 401
React Hook Form	7	Forms	Performant validation, minimal re-renders
Recharts	2	Charts	Radar + Bar charts for performance analytics
React Webcam	7	Camera	Webcam feed for anti-cheat monitoring
Web Speech API	Browser	Voice	Reads questions aloud — no library needed

Backend

Technology	Purpose	Why Chosen
Python Flask	REST API framework	Lightweight, easy to extend, great for AI integration
Flask-JWT-Extended	Authentication tokens	Industry standard, access + refresh token pattern
SQLAlchemy ORM	Database abstraction	Prevents SQL injection, DB-agnostic code
Flask-Mail	OTP email delivery	Integrates directly with Gmail SMTP
Flask-CORS	Cross-origin requests	Allows React frontend to call Flask API
Flask-Limiter	Rate limiting	Prevents brute force on auth endpoints
bcrypt	Password hashing	12 rounds — industry standard security
pdfplumber	PDF text extraction	Accurate text + layout extraction from resume PDFs
python-docx	DOCX extraction	Read Word document resume files
ReportLab	PDF report generation	Programmatic PDF creation with full formatting
Gunicorn	Production WSGI server	Multi-worker concurrent request handling

3. System Architecture — How Everything Connects

The application follows a classic 3-tier architecture separated into Frontend, Backend, and Database layers hosted on three different free cloud services.



Request Flow (what happens when you click something):

1. User performs an action in React (e.g. clicks Login button)
2. React calls Axios which adds the JWT token to the request header automatically
3. Request goes over HTTPS to the Flask API on Render
4. Flask route handler validates the JWT, checks permissions
5. SQLAlchemy queries the MySQL database on Clever Cloud
6. For AI tasks (questions/feedback), Flask calls Anthropic Claude API
7. Flask returns JSON response with status code
8. React updates state, re-renders the UI with new data
9. User sees the result — all in under 500ms for DB queries

4. Project Folder Structure

File / Folder	Purpose
frontend/src/pages/	All 11 page components — each maps to a URL route
frontend/src/components/common/	Shared components: Navbar used on all authenticated pages
frontend/src/context/AuthContext.js	Global user state + login/logout functions using React Context API
frontend/src/services/api.js	Axios instance with JWT interceptor — auto-attaches token to every request, auto-refreshes on 4
frontend/src/index.css	Tailwind CSS imports + Google Fonts (DM Sans, Syne)
frontend/tailwind.config.js	Custom colors (primary=indigo), fonts, animations config
frontend/vercel.json	Tells Vercel to redirect all routes to index.html for React Router
backend/app/__init__.py	Flask app factory — creates app, registers all blueprints, JWT error handlers
backend/app/config.py	Dev/Prod/Test configuration classes — reads from .env
backend/app/models/__init__.py	All 14 SQLAlchemy ORM models with relationships
backend/app/routes/auth.py	Auth endpoints: register, login, OTP, password reset
backend/app/routes/resume.py	Resume upload, NLP parsing, skill extraction
backend/app/routes/interview.py	Interview setup, questions, answer submission, scoring
backend/app/routes/results.py	Results retrieval with Q&A review
backend/app/routes/reports.py	PDF generation with ReportLab
backend/app/routes/dashboard.py	Stats and score trend for dashboard
backend/app/routes/admin.py	Admin-only user management, analytics
backend/app/services/ai_service.py	Claude API calls + local fallback — question generation + feedback
backend/app/utils/security.py	bcrypt password hash/verify + email/password validators
backend/run.py	Entry point — creates app, Flask CLI commands (init-db, seed-db)
backend/Procfile	Render start command: gunicorn -w 2 -b 0.0.0.0:\$PORT run:app
database/schema_fixed.sql	Complete MySQL schema — 14 tables, indexes, foreign keys, seed data

5. Database Schema — All 14 Tables

MySQL 8 on Clever Cloud. All tables use InnoDB with foreign key constraints and cascading deletes. JSON columns store flexible data like skills arrays and feedback objects.

users — Core — stores all student and admin accounts. Columns: id, full_name, email, password_hash (bcrypt), role (student/admin), is_active, is_verified, last_login.

otp_tokens — Stores 6-digit OTPs for email verification and password reset. Has expires_at timestamp. is_used flag prevents replay attacks. FK → users.

resumes — Stores uploaded resume metadata + NLP-parsed data in JSON columns: extracted_skills, extracted_projects, extracted_education, extracted_experience. parsed_text holds raw text. FK → users.

skills — Global skill library with categories (technical/soft/domain/tool). Referenced for tagging.

question_bank — Admin-managed question pool. Stores MCQs with options A-D, correct answer, explanation, difficulty, skill_tags (JSON), job_roles (JSON). FK → users (created_by).

interviews — One row per interview session. Stores job_role, job_description, experience_level, status (setup/technical_round/hr_round/completed), scores, violation_count, auto_submitted flag. FK → users + resumes.

interview_questions — Generated MCQ questions for each interview. Stores all 4 options, correct_answer, skill_tag, question_type (technical/hr), round_number. FK → interviews.

interview_answers — Student's answer for each question. selected_answer (A/B/C/D/skipped), is_correct (boolean), time_taken_seconds. FK → interviews + interview_questions.

interview_results — Calculated scores: correct/wrong/skipped counts for each round, percentages, overall_percentage, grade (A+ to F), skill_scores (JSON: {topic: {correct, total, percentage}}). FK → interviews (unique).

feedback — AI-generated feedback stored in JSON columns: strengths, weaknesses, technical_gaps, resume_suggestions, learning_roadmap. overall_summary as TEXT. confidence_score as DECIMAL. FK → interviews (unique).

violations — Anti-cheat log. violation_type ENUM (tab_switch, camera_off, copy_attempt, etc.), description, occurred_at. FK → interviews + users.

reports — PDF report metadata: report_path (file path on server), report_size, download_count. FK → interviews + users (unique).

badges — Gamification badge definitions: badge_name, badge_icon (emoji), condition_type, condition_value.

user_badges — Many-to-many: which badges each user has earned. earned_at timestamp. UNIQUE KEY prevents duplicates. FK → users + badges.

Key Relationships:

users ■■■(1:many)■■■■ resumes ■■■(1:many)■■■■ interviews

interviews ■■■(1:many)■■■■ interview_questions ■■■(1:many)■■■■ interview_answers

interviews ■■■(1:1)■■■■ interview_results

interviews ■■■(1:1)■■■■ feedback

interviews ■■■(1:1)■■■■ reports

interviews ■■(1:many)■■■ violations
users ■■(many:many)■■■ badges (via user_badges)

6. Backend — Flask API Deep Dive

How Flask is structured: The app uses the Application Factory pattern in `app/__init__.py`. This means `create_app()` is called with a config name, which allows different configs for development, production, and testing. All extensions (SQLAlchemy, JWT, Mail, CORS, Limiter) are initialized on the app object.

Blueprints: Each route file registers as a Blueprint with its own URL prefix. This keeps code modular — auth routes are separate from interview routes.

AUTH ROUTES — `/api/auth`

	What It Does	Auth Required
	Validates input → hashes password with bcrypt → creates user → generates 6-digit OTP → saves to otp_tokens table → sends email	No
	Checks OTP against otp_tokens table, verifies not expired (10 min), marks is_used=True, sets user.is_verified=True	No
	Deletes old unused OTPs for user → generates new OTP → emails it	No
	Checks email/password with bcrypt.checkpw → verifies is_active + is_verified → creates JWT access token (2hr) + refresh token	No
	Takes refresh token from header → generates new access token	Refresh JWT
	Finds user by email → deletes old reset OTPs → creates new OTP → emails it (15 min expiry)	No
	Validates OTP → validates new password strength → updates password_hash	No
	Decodes JWT → returns user object from DB	JWT
	Updates full_name and phone for current user	JWT

RESUME ROUTES — `/api/resume`

	What It Does
	Validates file type (PDF/DOCX only) + size (≤10MB) → saves with unique filename
	Returns all resumes for current user, ordered by upload date
	Returns specific resume with all extracted JSON data

INTERVIEW ROUTES — `/api/interview`

	What It Does
	Creates interview record → calls ai_service.generate_interview_questions()
	Returns questions for 'technical' or 'hr' round WITHOUT correct answers
	Saves each answer to interview_answers table → checks correctness
	Calls _calculate_scores() → sets status=completed → calls generate_feedback()
	Manually triggers feedback generation — useful when Claude API fails
	Logs violation type to violations table → increments interview.violations
	Returns all interviews for current user, newest first

Score Calculation (`_calculate_scores` function):

1. Queries all technical and HR questions for the interview
2. For each question, finds the corresponding answer in interview_answers
3. Counts correct / wrong / skipped separately for each round
4. $\text{technical_percentage} = (\text{correct_technical} / \text{total_technical}) \times 100$

5. $hr_percentage = (correct_hr / total_hr) \times 100$
6. $overall = (technical_pct + hr_pct) / 2$
7. Grade: A+(≥ 90), A(≥ 80), B+(≥ 70), B(≥ 60), C(≥ 50), D(≥ 40), F(<40)
8. Builds skill_scores dict: { 'React Hooks': {correct:2, total:3, percentage:66.7} }
9. Saves all to interview_results table

RESULTS ROUTE — /api/results/:id

Returns a single combined JSON object with: interview details, result scores, feedback (all 7 sections), full Q&A; review (questions with correct + selected answers), violations list. This is what the Results page renders.

DASHBOARD ROUTE — /api/dashboard/stats

Queries all completed interviews for the user → calculates averages → returns: total_interviews, completed_interviews, avg_score, avg_technical, avg_hr, recent_interviews (last 6), score_trend (last 10 scores with dates for the line chart).

ADMIN ROUTES — /api/admin (admin role required)

Endpoint	What It Does
/users	Paginated list of all users with all fields
/users/:id/toggle	Flip user.is_active between True/False
/interviews	Paginated list of all interviews across all users
/violations	Last 100 violations across all interviews
/analytics	Total users, interviews, avg score, completion rate
/questions (POST)	Add question to question_bank table

7. Frontend — All 11 Pages Explained

Routing: React Router v6. Protected routes check AuthContext for user object. If no user → redirect to /login. Admin-only routes also check user.role === 'admin'.

Auth State: `AuthContext.js` stores the logged-in user globally. On app load, it reads the JWT from localStorage, calls `/api/auth/me` to verify it, and sets the user state. If the token is invalid, it clears localStorage.

Axios Interceptor: `api.js` attaches 'Authorization: Bearer {token}' to every request. If a request returns 401, it auto-calls `/api/auth/refresh` with the refresh token to get a new access token, then retries the original request.

HomePage /

Public landing page. Shows features, how-it-works steps, and CTA buttons. No auth required. Uses Framer Motion for scroll animations.

LoginPage /login

Email + password form using React Hook Form. Calls POST `/api/auth/login`. Stores `access_token` and `refresh_token` in localStorage. Remember Me sets 30-day token. Redirects to `/dashboard` on success.

RegisterPage /register

2-step: Step 1 = registration form with strong password validation. Step 2 = OTP entry box. Calls POST `/api/auth/register` → then POST `/api/auth/verify-email`. Has Resend OTP button calling POST `/api/auth/resend-otp`.

ForgotPasswordPage /forgot-password

3-step flow: Step 1 = enter email (calls `/forgot-password`). Step 2 = enter OTP + new password (calls `/reset-password`). Step 3 = success message with link to login.

DashboardPage /dashboard

After login. Fetches GET `/api/dashboard/stats`. Shows 4 stat cards, score trend line chart (Recharts), performance split donut (SVG), recent interviews table with links.

StartInterviewPage /interview/start

Drag-drop resume upload → calls POST `/api/resume/upload` → shows detected skills on success. Job role, experience level dropdown, job description textarea. Submit calls POST `/api/interview/setup` → navigates to `/interview/:id`.

InterviewPage /interview/:id

The core interview screen. Fetches questions for current round. Timer counts down 20s per question using `setInterval`. Web Speech API reads each question + options aloud. Anti-cheat hooks block right-click, copy, paste, F12, tab-switch. Webcam renders in top bar. Answers auto-advance. After all questions, calls POST `/api/interview/:id/submit-round`. After HR round, calls POST `/api/interview/:id/complete`.

ResultsPage /results/:id

Fetches GET `/api/results/:id`. Four tabs: Overview (radar chart + bar chart + round breakdown), Review (Q&A; with correct/wrong highlighted), Feedback (7 sections), Violations. Voice button reads feedback aloud. Download PDF button calls POST `/reports/generate/:id` then GET `/reports/download/:id`.

HistoryPage /history

Fetches GET /api/interview/history. Filters by status. Shows score delta vs previous interview. Links to Results or Resume-interview button.

ProfilePage /profile

Update name/phone via PUT /api/auth/update-profile. Change password: first get OTP via forgot-password, enter OTP + new password → POST /api/auth/reset-password.

AdminPage /admin

4 tabs: Overview (analytics cards + recent violations), Users (table with activate/deactivate toggle), Interviews (all interviews table), Violations (all violation log). Admin-only route.

8. Authentication Flow — Step by Step

REGISTRATION:

1. User fills form → React Hook Form validates (email format, password strength: 8+ chars, uppercase, lowercase, number, special char)
2. POST /api/auth/register → Flask validates → bcrypt.hashpw(password, salt) → creates User row
3. Flask generates 6-digit random OTP → saves to otp_tokens table with expires_at = now + 10 minutes
4. Flask-Mail sends HTML email via Gmail SMTP with OTP
5. User enters OTP → POST /api/auth/verify-email → Flask checks token, expiry, is_used flag
6. Sets user.is_verified = True → user can now login

LOGIN:

1. POST /api/auth/login → Flask finds user by email → bcrypt.checkpw(password, hash)
2. Checks is_active + is_verified flags
3. create_access_token(identity=user.id, expires=2hr) + create_refresh_token(expires=30d)
4. Frontend stores both tokens in localStorage
5. Axios interceptor adds 'Authorization: Bearer {access_token}' to every future request

TOKEN REFRESH:

1. Access token expires after 2 hours
2. Axios intercepts 401 response → calls POST /api/auth/refresh with refresh token
3. Flask generates new access token → Axios retries the original request
4. User never notices — seamless experience

LOGOUT:

1. Frontend clears localStorage (access_token, refresh_token)
2. AuthContext sets user = null
3. React Router redirects to /login
4. Tokens are stateless — no server-side session to invalidate

9. Interview Flow — End to End

STEP 1 — Resume Upload

- User drops PDF/DOCX on the upload zone in StartInterviewPage
- File validated: type must be pdf/docx, size $\leq$ 10MB
- Saved to /uploads/ folder with UUID filename to avoid conflicts
- pdfplumber opens PDF, extracts text page by page
- NLP parser: scans text against 60+ known tech skills using regex word boundaries
- Section parser: looks for headers like 'Skills', 'Experience', 'Projects', 'Education'
- Saves parsed data as JSON in resumes table

STEP 2 — Interview Setup

- User enters: Job Role, Job Description (paste from job posting), Experience Level
- POST /api/interview/setup → creates interview row
- Calls ai_service.generate_interview_questions() with all resume data
- If Claude API has credits: sends detailed prompt to claude-haiku-4-5 model
- Claude returns JSON with 15 technical + 10 HR MCQs
- If Claude fails: uses role-aware fallback pool (data science / web / backend / general)
- All questions saved to interview_questions table

STEP 3 — Technical Round

- InterviewPage fetches GET /api/interview/:id/questions/technical
- Backend returns questions WITHOUT correct_answer field
- Browser enters fullscreen mode (requestFullscreen API)
- Web Speech API reads each question + 4 options aloud
- 20-second countdown per question (setInterval)
- Student selects answer → answer saved to state → auto-advances to next question
- If timer hits 0 → question marked as 'skipped' → advances
- After last question → POST /api/interview/:id/submit-round (round_type: technical)
- Backend saves all answers to interview_answers table with is_correct flag

STEP 4 — HR Round

- Same flow as Technical Round but with HR/behavioral questions
- POST /api/interview/:id/submit-round (round_type: hr)

STEP 5 — Completion

- POST /api/interview/:id/complete
- _calculate_scores() runs: counts correct/wrong/skipped per round, calculates percentages, assigns grade, builds skill_scores dict
- Saves to interview_results table
- generate_feedback() is called: queries wrong answers, builds context, calls Claude API
- If Claude fails: generate_feedback_local() creates rule-based feedback from scores

- Feedback saved to feedback table in JSON columns
- Frontend navigates to /results/:id

STEP 6 — Results

- GET /api/results/:id returns everything in one call
- ResultsPage renders 4 tabs: Overview (charts), Review (Q&A;), Feedback (7 sections), Violations
- Voice button uses SpeechSynthesisUtterance to read feedback aloud
- Download PDF → POST /reports/generate/:id → ReportLab builds PDF → GET /reports/download/:id → file streamed to browser

10. Anti-Cheat System

The anti-cheat system runs entirely in the browser during the interview using JavaScript APIs. All violations are logged to the backend.

Violation Type	Detection Method	What Happens
Tab Switch / Window Minimize	document.addEventListener('visibilitychange', function() { if (document.visibilityState === 'hidden') { logViolation('tab_switch'); } });	Logs tab switch violation. Backend, when scrolling back, then becomes true
Right Click	document.addEventListener('contextmenu', function(e) { e.preventDefault(); logViolation('right_click'); });	Context menu blocked, logs 'right_click' violation
Copy Attempt	document.addEventListener('copy', function(e) { e.preventDefault(); logViolation('copy_attempt'); });	Clipboard blocked, logs 'copy_attempt'
Paste Attempt	document.addEventListener('paste', function(e) { e.preventDefault(); logViolation('paste_attempt'); });	Paste blocked, logs 'paste_attempt'
Developer Tools	document.addEventListener('keydown', function(e) { if (e.keyCode === 123 e.ctrlKey e.shiftKey e.altKey) { logViolation('dev_tools'); } });	Key blocks F12, Ctrl, Shift, Alt, shortcut
Fullscreen Exit	document.addEventListener('fullscreenchange', function() { if (!document.fullscreenElement) { logViolation('fullscreen_exit'); } });	Logs if user presses 'Esc' to exit fullscreen. If escape browser allows exit
Camera Off	React Webcam onUserMediaError callback logs error if camera is not available	Logs 'camera_off' if camera is not available
Auto Submit	violation_count increments on each violation	After 10 violations: interview.auto_submitted=True, scores calculated, interview ends

11. AI Service — Questions & Feedback

File: backend/app/services/ai_service.py

generate_interview_questions() — How it works:

1. Receives: job_role, job_description, experience_level, skills[], projects[], education[], resume_text
2. Builds a detailed prompt string (NOT an f-string to avoid JSON brace conflicts) with all candidate data
3. Calls `_call_claude()` → POST to <https://api.anthropic.com/v1/messages> with model `claude-haiku-4-5`
4. Parses JSON response: expects `{technical: [...15 questions...], hr: [...10 questions...]}`
5. Each question has: question, options (A/B/C/D), correct, skill_tag, explanation
6. If Claude fails → uses `_fallback_technical()` and `_fallback_hr()` — role-aware randomized pools
7. Fallback pools: data/analytics roles get Pandas/SQL/ML questions; web roles get React/JS/Flask; generic gets DSA/OOP

generate_feedback() — How it works:

1. Queries all questions + answers for the interview, builds skill_performance dict
2. Identifies weak topics (more wrong than correct) and strong topics
3. Builds prompt using string concatenation (not f-string) — JSON schema as plain string
4. Calls Claude API for personalized analysis
5. If Claude fails → `generate_feedback_local()` uses pure Python logic based on scores
6. Local feedback: generates strengths/weaknesses from score thresholds, tech gaps from wrong answers, role-specific missing skills, 4-week study plan, curated resources
7. Saves to feedback table: strengths/weaknesses/technical_gaps as JSON arrays, resume_suggestions and learning_roadmap as nested JSON objects

_call_claude() — The API call:

- Reads `ANTHROPIC_API_KEY` from environment
- POST to <https://api.anthropic.com/v1/messages>
- Headers: `x-api-key`, `anthropic-version: 2023-06-01`
- Body: `model`, `max_tokens`, `messages` array
- Returns `response.json()[content][0][text]`
- Raises `RuntimeError` with HTTP status if not 200 — visible in Render logs

12. PDF Report Generation

Library: ReportLab (Python) | **File:** backend/app/routes/reports.py

How it works:

1. POST /api/reports/generate/:id → fetches interview + result + feedback + violations from DB
2. Creates SimpleDocTemplate with A4 page size, margins
3. Builds story list (list of flowable objects: Paragraphs, Tables, Spacers, HRFlowable)
4. Candidate Information table: name, email, job role, experience, date, status
5. Performance Summary table: technical score, HR score, overall + grade
6. AI Feedback sections: overall summary, communication analysis, strengths, weaknesses, technical gaps, missing skills, resume improvements, focus areas, weekly study plan, interview tips
7. Violations table (if any): type, description, timestamp
8. doc.build(story) generates the PDF
9. Saves to /reports/report_{interview_id}_{user_id}.pdf
10. Stores path + file_size in reports table
11. GET /api/reports/download/:id → Flask send_file() streams PDF to browser with attachment header

13. Deployment Architecture

Frontend	Vercel — auto-deploys from GitHub main branch. Root dir: frontend/. Build: npm run build. Output: build/. vercel
Backend	Render Web Service — auto-deploys from GitHub. Root dir: backend/. Build: pip install -r requirements.txt. Star
Database	Clever Cloud MySQL DEV plan — 10MB free. phpMyAdmin available. Connection via individual DB_HOST/DB_
DNS/HTTPS	Both Vercel and Render provide automatic HTTPS with SSL certificates. No configuration needed.
CORS	Flask-CORS restricts allowed origins to CORS_ORIGINS env var. Set to https://ai-interview-portal-roni.vercel.a
File Storage	Uploaded resumes and generated PDFs stored on Render's ephemeral filesystem. Files lost on redeploy. For p

Environment Variable Flow:

Local dev: reads from backend/.env file via python-dotenv load_dotenv()

Production: Render injects env vars directly into the process environment — no .env file needed

Frontend: Vercel injects REACT_APP_* vars at build time into the JavaScript bundle

14. Environment Variables Reference

Variable	Example Value	Purpose	Where Set
FLASK_ENV	production	Switches between Dev/Prod config	Render
SECRET_KEY	random-string-abc	Flask session signing key	Render
JWT_SECRET_KEY	random-string-xyz	JWT token signing key	Render
DB_HOST	bxxxxx.mysql.clevercloud.com	Clever Cloud MySQL host	Render
DB_PORT	3306	MySQL port	Render
DB_USER	unkxez6...	Clever Cloud username	Render
DB_PASSWORD	Xm22oFn...	Clever Cloud password	Render
DB_NAME	bsrproeej...	Clever Cloud database name	Render
MAIL_SERVER	smtp.gmail.com	Gmail SMTP server	Render
MAIL_PORT	587	SMTP TLS port	Render
MAIL_USERNAME	yourmail@gmail.com	Gmail account (must match sender)	Render
MAIL_PASSWORD	xxxx xxxx xxxx xxxx	Gmail App Password (16 chars)	Render
CORS_ORIGINS	https://app.vercel.app	Allowed frontend origins	Render
ANTHROPIC_API_KEY	sk-ant-api03-...	Claude AI API key	Render
REACT_APP_API_URL	https://xxx.onrender.com	Backend URL for Axios	Vercel

15. How to Make Changes — Developer Guide

COMMON INTERVIEW REQUESTS — HOW TO IMPLEMENT EACH:

→ Change the number of interview questions

1. Open: `backend/app/services/ai_service.py`
2. In `generate_interview_questions()` prompt, change 'Generate exactly 15 technical MCQ' to your number
3. Change the slice at the end: `technical[:15]` → `technical[:YOUR_NUMBER]`
4. Also change the fallback: `pool[:15]` → `pool[:YOUR_NUMBER]`
5. In `InterviewPage.js` nothing needs to change — it reads the count dynamically

→ Change the question timer (currently 20 seconds)

1. Open: `backend/app/config.py` → change `QUESTION_TIME_LIMIT = 20` to your value
2. Open: `backend/app/routes/interview.py` → in `setup_interview()`, change `time_limit_seconds=20`
3. Open: `frontend/src/pages/InterviewPage.js` → change `setTimeLeft(20)` and `(timeLeft / 20) * 100` to your value

→ Add a new page / route

1. Create: `frontend/src/pages/NewPage.js` (copy structure from `HistoryPage.js`)
2. Add route in `frontend/src/App.js`: `} />`
3. Add import at top of `App.js`: `import NewPage from './pages/NewPage'`
4. Add link in `frontend/src/components/common/Navbar.js` `navLinks` array

→ Add a new API endpoint

1. Open the relevant route file in `backend/app/routes/`
2. Add a new function with `@blueprint.route('/endpoint', methods=['GET'])` decorator
3. Use `@jwt_required()` decorator if auth is needed
4. Access current user: `user_id = get_jwt_identity()`
5. Return: `return jsonify({'data': ...}), 200`
6. No need to register — blueprints auto-include all routes in the file

→ Add a new database column

1. Open: `backend/app/models/__init__.py`
2. Find the relevant model class and add: `new_field = db.Column(db.String(100), default="")`
3. In phpMyAdmin on Clever Cloud: `ALTER TABLE table_name ADD COLUMN new_field VARCHAR(100) DEFAULT ""`
4. Update the model's `to_dict()` method to include the new field
5. Update relevant route to read/write the new field

→ Change the color theme

1. Open: `frontend/tailwind.config.js`
2. Change the primary color hex values under `theme.extend.colors.primary`

3. All components use 'primary-500', 'primary-600' etc. — they all update automatically
4. Run: `npm start` to see the change instantly

→ Add Google OAuth login

1. Install: `pip install flask-dance` (backend), `npm install @react-oauth/google` (frontend)
2. Create Google OAuth app at console.developers.google.com
3. Add route: `GET /api/auth/google` → flask-dance handles OAuth flow
4. In frontend `LoginPage.js`: add `GoogleOAuthProvider` wrapper and `useGoogleLogin` hook
5. On success: Google returns user email → find/create user in DB → return JWT

→ Store files on AWS S3 (instead of local filesystem)

1. Install: `pip install boto3`
2. Add env vars: `AWS_ACCESS_KEY`, `AWS_SECRET_KEY`, `S3_BUCKET_NAME`
3. In `resume.py`: after `file.save()` → use `boto3` to upload to S3 → store S3 URL in DB
4. In `reports.py`: after PDF generation → upload to S3 → return S3 URL for download
5. This fixes the issue of files being lost on Render redeploy

→ Change violation limit (currently 3 violations = auto submit)

1. Open: `backend/app/config.py` → change `MAX_VIOLATIONS_BEFORE_SUBMIT = 3` to your value
2. The frontend warning messages in `InterviewPage.js` also reference this — update the `VIOLATION_MAX` constant

→ Add email notifications after interview completion

1. Open: `backend/app/routes/interview.py` → in `complete_interview()` function
2. After `generate_feedback()` succeeds, call a new function `send_completion_email(user, interview, result)`
3. Create the function in `auth.py` or a new `utils/email.py` file using Flask-Mail
4. Include score, grade, and link to results page in the email HTML

16. Common Interview Questions & Answers

Use these to explain the project confidently to any interviewer:

Q: Why did you choose React over Angular or Vue?

A: React's component model and one-way data flow made it ideal for this project. The Context API handles global auth state cleanly. React's virtual DOM ensures smooth updates during the interview timer. Also, the ecosystem (Recharts, Framer Motion, React Hook Form) saved significant development time.

Q: How does JWT authentication work in your project?

A: When a user logs in, the Flask backend creates two tokens: an access token (2 hours) signed with `JWT_SECRET_KEY`, and a refresh token (30 days). The frontend stores both in `localStorage`. Axios automatically attaches the access token to every request. When the access token expires, the Axios interceptor catches the 401 response, calls `/api/auth/refresh` with the refresh token, gets a new access token, and retries the original request. The user never needs to log in again for 30 days.

Q: How does the resume parsing work?

A: When a user uploads a PDF, `pdfplumber` opens it and extracts text from each page. For DOCX files, `python-docx` reads paragraph text. Then a custom NLP function scans the extracted text against a list of 60+ known tech skills using Python regex with word boundaries to avoid false matches. A section parser splits the text by headers like 'Skills', 'Experience', 'Projects' to organize extracted data. All results are stored as JSON columns in the resumes table.

Q: How do you prevent cheating during the interview?

A: I implemented a multi-layer anti-cheat system using browser APIs. The Visibility API detects tab switches. Right-click and keyboard shortcuts (F12, Ctrl+Shift+I) are blocked via event listeners. The browser enters fullscreen mode. A webcam feed monitors the student. All violations are logged to the database with timestamps. After 3 violations, the interview auto-submits. Each violation type has its own ENUM value in the database.

Q: What happens when the Claude API is unavailable?

A: I implemented a two-layer fallback. First, if the Claude API returns an error (credit limit, network issue, wrong model name), the exception is caught and logged. Then `generate_feedback_local()` is called which uses pure Python logic: it analyses the actual correct/wrong answers, identifies weak topics from wrong answer `skill_tags`, generates role-specific feedback based on the job role, creates a 4-week study plan with curated resources, and saves everything in the same format. The user gets complete feedback regardless.

Q: Why MySQL over MongoDB for this project?

A: This project has clearly relational data: users have resumes, resumes belong to interviews, interviews have questions, questions have answers, results belong to interviews. All these relationships benefit from foreign keys and JOIN queries. The `interview_results` table has a UNIQUE constraint on `interview_id` — impossible in a document store. MySQL's ENUM types for `violation_type` and `grade` also enforce data integrity. I used JSON columns for flexible data like skill arrays and feedback objects.

Q: How does the score calculation work?

A: The `_calculate_scores()` function in `interview.py` queries all questions for the interview, then for each question finds the corresponding answer in `interview_answers`. It counts correct/wrong/skipped separately for

technical and HR rounds. $\text{technical_percentage} = (\text{correct_count} / \text{total_count}) \times 100$. $\text{overall_score} = (\text{technical_pct} + \text{hr_pct}) / 2$. A grade is assigned: A+ (≥ 90), A (≥ 80), B+ (≥ 70), B (≥ 60), C (≥ 50), D (≥ 40), F (< 40). A `skill_scores` dict is built mapping each `skill_tag` to its correct/total/percentage.

Q: How do you handle file uploads securely?

A: File type is validated by checking the extension (PDF/DOCX only) — not just the MIME type which can be spoofed. File size is limited to 10MB via Flask's `MAX_CONTENT_LENGTH` config. Filenames are sanitized with `werkzeug's secure_filename()` and prefixed with `user_id + UUID` to prevent path traversal attacks and filename collisions. Files are stored outside the web-accessible directory.

Q: Explain the database connection setup for production.

A: Locally I use individual `DB_HOST/DB_USER/DB_PASSWORD` env vars to build the SQLAlchemy connection string. On Clever Cloud, I use the same individual vars set in Render's environment. The `config.py` checks for a `DATABASE_URL` env var first (some platforms like Railway provide this), fixes the `mysql://` prefix to `mysql+pymysql://`, then falls back to building from individual vars. I set `pool_pre_ping=True` so SQLAlchemy automatically reconnects if the Clever Cloud connection drops.

Q: How would you scale this application?

A: Currently: single Gunicorn server with 2 workers. To scale: (1) Move file storage to AWS S3 — current ephemeral filesystem loses files on redeploy. (2) Add Redis for rate limiting instead of in-memory (works across multiple servers). (3) Move feedback generation to a background task queue (Celery + Redis) so the API doesn't block waiting for Claude. (4) Add read replicas for the MySQL database. (5) Use a CDN for the React static assets. (6) Upgrade Render to a paid tier to eliminate cold starts.

